

Atividade Prática Munif

Rafael Blasques Morales
24124872-2

2 - Slide:

Objetivo e ligação com o mundo real

Objetivo

O objetivo desta aula é entender **como as linguagens avaliam expressões e controlam o fluxo de execução**, para que o seu código faça exatamente o que parece fazer, **sem surpresas** de precedência, conversão ou desvio.

Para o engenheiro de software, isso significa:

- ler código alheio sem ser enganado pela endentação ou pela precedência
- evitar conversões implícitas perigosas (`==` do JS e do PHP)
- escolher a estrutura de controle mais legível para cada caso

Quando o fluxo sai do controle

2014 · "goto fail" (Apple)

Uma linha `goto fail`; duplicada pulava a verificação de assinatura: iOS e macOS aceitavam certificados falsos.

2008 · Tocadores Zune (Microsoft)

No último dia de um ano bissexto, um laço `while` nunca terminava, e todos os aparelhos travaram.

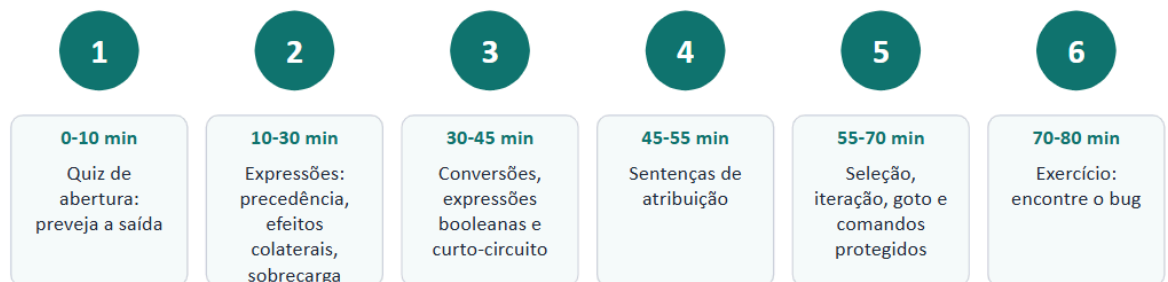
PHP · comparação com `==`

`"0e4..." == "0e8..."` é verdadeiro: logins que comparavam hashes com `==` aceitavam senhas erradas.

Compreendi o real objetivo da aula. Entender como o fluxo de execução e como as linguagens são avaliadas é de suma importância para um engenheiro de software. Quando um fluxo sai do controle, erros cruciais podem acontecer e gerar prejuízos inimagináveis.

3 - Slide:

Roteiro da aula (80 min)



Tipo de aula: exploratória. A turma **prevê a saída antes de executar**.

Exemplos: pasta `exemplos/`, na ordem dos slides. Todos rodam no navegador em **onecompiler.com**.

Apenas o roteiro da aula.

4 - Slide:

Quiz de abertura: o que cada linha imprime?

Anote o seu palpite para cada trecho **antes** de executar. Vamos conferir ao longo da aula.

1 - Python

```
print(-2 ** 2, 2 ** 3 ** 2)
```

2 - Java

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

3 - JavaScript

```
console.log("5" - 2, "5" + 2);
```

4 - C

```
printf("%d\n", 3 > 2 > 1);
```

5 - Python

```
x = 0
print(x or "padrão", 3 > 2 > 1)
```

6 - JavaScript

```
console.log(0 == "", "" == "0", 0 == "0");
```

1 - Python: -4 512, será calculado as potências, 2^{**} e $2^{**3^{**2}}$.

2 - Java: 339, no println, para juntar as frases, é utilizado o +, por isso começa fazendo a soma, quando chega na string, imprime a string junto com o resultado anterior, e depois continua somando.

3 - JavaScript: 5252, meu palpite é que não será somado e será exibido dessa forma.

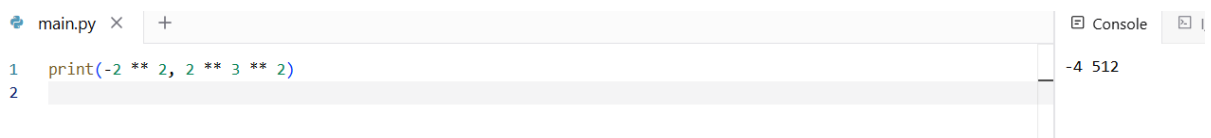
4 - C: 3, meu palpite é que os sinais de ">" não vão interferir e será printado o primeiro número

5 - Python: padrão3, meu palpite é que a linguagem vai considerar x=0 e vai printar apenas o "padrão" e o 3 por ser o primeiro número na sequência.

6 - true true true, meu palpite é que a linguagem considere tudo como verdadeiro.

RESULTADO:

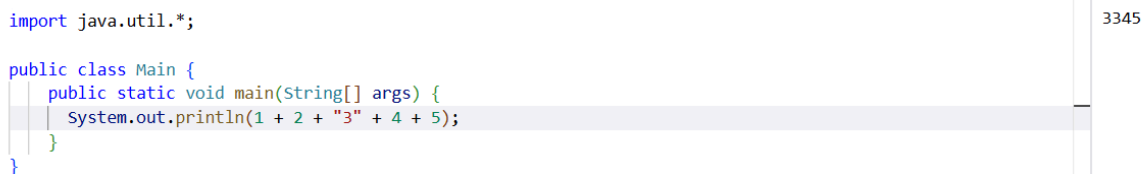
1



```
main.py x +
1 print(-2 ** 2, 2 ** 3 ** 2)
2
```

Console -4 512

2

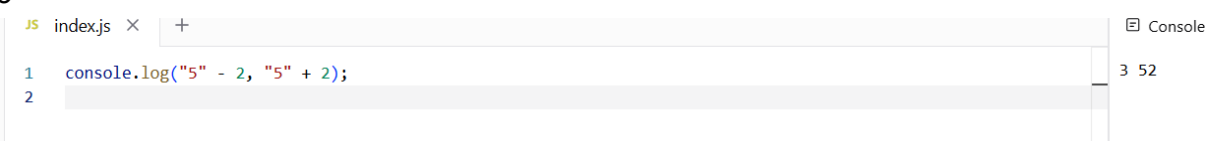


```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        System.out.println(1 + 2 + "3" + 4 + 5);
    }
}
```

3345

3



```
index.js x +
1 console.log("5" - 2, "5" + 2);
2
```

Console 3 52

4

```
#include <stdio.h>
int main()
{
    printf("%d\n", 3 > 2 > 1);
}
```

0

5

```
x = 0
print(x or "padrão", 3 > 2 > 1)
```

padrão True

6

```
console.log(0 == "", "" == "0", 0 == "0");
```

true false true

5 - Slide:

Expressões aritméticas: questões de projeto

- Quais são as regras de **precedência**?
- Quais são as regras de **associatividade**?
- Em que **ordem os operandos** são avaliados?
- Há restrições a **efeitos colaterais**?
- O usuário pode **sobrecarregar** operadores?
- Que **mistura de tipos** é permitida?

Lisp dispensa a precedência (p. 306)

(+ a (* b c)): notação prefixa, e os parênteses dizem a ordem.

a + b * c com a=3, b=4, c=5

da esquerda para a direita

(3 + 4) * 5 = 35

da direita para a esquerda

3 + (4 * 5) = 23

com precedência (* antes de +)

3 + 4 * 5 = 23

APL: sempre da direita para a esquerda (p. 304)

3 × 4 + 5 = 27

Sebesta, 11ª ed. - §7.2 e §7.2.1 - p. 300-305

1. Quais são as regras de precedência?

As regras de precedência determinam a prioridade dos operadores no agrupamento de uma expressão. Geralmente, multiplicação e divisão têm prioridade sobre adição e subtração. Por exemplo, $2 + 3 * 4$ resulta em 14, pois equivale a $2 + (3 * 4)$. Os parênteses permitem alterar esse agrupamento: $(2 + 3) * 4$ resulta em 20.

2. Quais são as regras de associatividade?

As regras de associatividade determinam como operadores de mesma precedência são agrupados quando não há parênteses. Esse agrupamento pode ocorrer da esquerda para a direita ou da direita para a esquerda, conforme o operador e a linguagem. Por exemplo, com associatividade à esquerda, $10 - 3 - 2$ equivale a $(10 - 3) - 2$, resultando em 5.

3. Em que ordem os operandos são avaliados?

A ordem de avaliação determina a sequência em que os valores dos operandos são obtidos. Na expressão `funcaoA() + funcaoB()`, essa regra define qual função será executada primeiro. Algumas linguagens estabelecem uma ordem, enquanto outras deixam essa sequência sem especificação. A precedência define o agrupamento dos operadores; a ordem de avaliação define a sequência de execução.

4. Há restrições a efeitos colaterais?

Efeitos colaterais são alterações de estado ou interações com o ambiente durante a avaliação de uma expressão, como modificar uma variável ou escrever em um arquivo. As restrições dependem da linguagem. Uma expressão que modifica e consulta a mesma variável pode ter comportamento definido, ser proibida ou apresentar comportamento indefinido, conforme as regras de avaliação e sequenciamento.

5. O usuário pode sobrecarregar operadores?

A sobrecarga de operadores depende dos recursos oferecidos pela linguagem. Quando permitida, o programador pode definir o comportamento de um operador para tipos criados por ele. Por exemplo, o operador `+` pode representar a soma das coordenadas de dois objetos do tipo `Vetor`. Normalmente, a sobrecarga preserva a precedência e a associatividade do operador.

6. Que mistura de tipos é permitida?

A mistura de tipos depende das regras de compatibilidade e conversão da linguagem. Uma operação entre um inteiro e um número de ponto flutuante pode realizar uma conversão automática para um tipo compatível. Outras combinações, como texto e número, podem exigir conversão explícita, realizar concatenação ou gerar erro. A linguagem também determina o tipo do resultado da operação.

6 - Slide:

Precedência e associatividade na prática

Expressão	Python	JavaScript	Java
<code>-2 ** 2</code>	-4	erro de sintaxe	(não há <code>**</code>)
<code>2 ** 3 ** 2</code>	512	512	–
<code>1 + 2 + "3"</code>	TypeError	"33"	"33"
<code>-7 / 2</code> (inteiros)	-4 (com <code>//</code>)	-3.5	-3
<code>-7 % 2</code>	1	-1	-1

Associatividade (p. 303)

Esquerda: `*` `/` `%` `+` `-`
Direita: `**` (Python, JS, Ruby, Fortran),
`++`, `--`, menos unário
Visual Basic: `^` à esquerda!

Excel

`=-2^2` dá 4: lá o menos unário vence o `^`.

Na dúvida, **use parênteses** (p. 305): eles deixam a intenção explícita para o compilador e para o próximo leitor.

Compreendi que diferentes linguagens podem interpretar a mesma operação de formas diferentes. Isso é a que as tornam diferentes, e essenciais para cada uso específico. A potenciação em python por exemplo é da direita pra esquerda, já em VisualBasica é à esquerda

7 - Slide:

Ordem dos operandos e efeitos colaterais

Exemplo do livro (p. 307)

```
int a = 5;

int fun1(void) {
    a = 17;      /* efeito colateral */
    return 3;
}

int main(void) {
    a = a + fun1(); /* 8 ou 20? */
    printf("a = %d\n", a);
}
```

8

Java

a ordem é garantida: da esquerda para a direita (p. 308)

20

C (gcc 14)

a ordem não é definida pelo padrão

20

Go

a especificação não define a leitura de `a` em relação à chamada

Transparência referencial (p. 308): sem efeitos colaterais, a ordem deixa de importar, e `f(x) + f(x)` pode virar `2 * f(x)`.

Sebesta, 11ª ed. · 67.2.2 · p. 307-309 · exemplos/02_efeitos_colaterais

O mesmo código pode retornar valores diferentes com base nas regras da linguagem. Java mantém a ordem garantida, e não permite alterar a variável direto na função. Por outro lado, C e GO permitem que isso aconteça.

8 - Slide:

Operadores sobrecarregados

- O mesmo símbolo com **vários significados**: + soma inteiros, reais e concatena (Java)
- Risco (p. 309): & em C++ é E bit a bit e "endereço de"
- **Definida pelo usuário** (p. 404): C++, C#, Python, Rust, Kotlin. Java **não** permite

Java

```
System.out.println('a' + 1);           // 98
System.out.println("'" + 'a' + 1);    // a1
x.add(y).multiply(tres);              // BigDecimal: sem +
```

Python - sobrecarga com métodos especiais

```
class Vetor:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, outro):          # v1 + v2
        return Vetor(self.x + outro.x,
                       self.y + outro.y)

    def __mul__(self, k):               # v * 3
        return Vetor(self.x * k, self.y * k)

v = Vetor(1, 2) + Vetor(3, 4)         # Vetor(4, 6)
print(v * 3)                          # Vetor(12, 18)
```

Sebesta, 11ª ed. - §7.3 - p. 309-310 e §9.11 - p. 404 - exemplos/03_sobrecarga

Algumas linguagens permitem sobrecarregar operadores, de jeitos únicos em cada uma. Python usa métodos especiais (**__add__**), mas Java decidiu não permitir essa possibilidade, cujo o "+" serve apenas para inteiros, reais e concatenação.

9 - Slide:

Conversões de tipos: alargamento, estreitamento e coerção

Java - coerção e cast

```
int grande = 123456789;
float f = grande;           // alargamento implícito
System.out.println((int) f); // 123456792 !

double d = 3.99;
System.out.println((int) d); // 3 (cast)
System.out.println((byte) 200); // -56

byte b = 10, c = 20;
// byte s = b + c; // erro: b + c é int (p. 312)

System.out.println(1 / 2);    // 0
System.out.println(1 / 2.0);  // 0.5
```

Linguagem	int → real (implícito)	int32 + int64
C	sim	coerção
Java	sim (alargamento)	coerção para long
Python	sim	(um só tipo int)
Go	não : float64(i)	erro de compilação
Rust	não : i as f64	erro de compilação
ML, F# (p. 312)	não	erro de compilação

Coerção esconde erros (p. 312): em `d = b * a`; com `a` digitado por engano no lugar de `c`, o Java converte e compila.

Compreendi que as conversões de tipos podem ser automáticas ou explícitas, dependendo da linguagem. Mesmo o alargamento pode causar perda de precisão, enquanto converter um número real para inteiro pode descartar as casas decimais, como 3,99 virar 3. Java permite algumas conversões automáticas, já Go e Rust exigem conversões explícitas nos casos apresentados.

10 - Slide:

Coerção implícita: JavaScript e PHP

JavaScript

```
"5" - 2           // 3
"5" + 2           // "52"
"5" * "2"         // 10
true + 1          // 2
[] + {}           // "[object Object]"
null + 1          // 1
undefined + 1     // NaN
"b" + "a" + "a" + "a" // "baNaNa"

0 == ""          // true
"" == "0"        // false
0 == "0"         // true (não é transitivo!)
0 === "0"        // false (=== não converte)
```

PHP - "type juggling"

```
var_dump("0e123" == "0e456"); // true !
var_dump("0e123" === "0e456"); // false
var_dump("1" == "01"); // true

md5("240610708") // "0e462097431906..."
md5("QNKCDZO") // "0e830400451993..."
// == entre os dois hashes: true
```

Regra prática

Em JavaScript e PHP, compare com `===` e `!==`, que **não** fazem coerção.

Compreendi que a coerção implícita pode gerar resultados inesperados. Em JavaScript, `"5" - 2` resulta em 3, mas `"5" + 2` resulta em "52", pois o operador muda a interpretação. Tanto JavaScript quanto PHP podem converter tipos ao comparar com `==`. Por isso, utilizar `===` e `!==` ajuda a evitar comparações que considerem valores de tipos diferentes como iguais.

11 - Slide:

Expressões relacionais e booleanas

C - booleano é inteiro (p. 315)

```
int a = 3, b = 2, c = 1;
printf("%d\n", a > b > c); // 0 !
// (3 > 2) vale 1, e 1 > 1 é falso

printf("%d\n", (a > b) && (b > c)); // 1

if (-1) printf("verdadeiro\n"); // != 0
```

Python - comparações encadeadas

```
a, b, c = 3, 2, 1
print(a > b > c) # True: (a > b) and (b > c)

# contam como FALSO em um if:
# 0 0.0 "" [] {} None
# contam como VERDADEIRO:
# "0" [0] " "
```

Linguagem	O que o if aceita
C, Python, JavaScript	qualquer valor (0, vazio e nulo contam como falso)
Java, C#, Go, Rust	somente booleano: <code>if (n)</code> com n inteiro nem compila

Compreendi que comparações encadeadas podem funcionar de formas diferentes. Em C, `3 > 2 > 1` resulta em falso, pois a primeira comparação retorna 1, e depois `1 > 1` é falso. Python interpreta essa sequência como duas comparações ligadas por "e", resultando em verdadeiro. Além disso, C aceita números no if, considerando verdadeiro qualquer valor diferente de zero, enquanto Java, Go e Rust exigem uma condição booleana.

12 - Slide:

Avaliação em curto-circuito

Java · && faz curto-circuito, & não

```
// busca do livro (p. 316)
while (i < lista.length && lista[i] != chave)
    i++;           // && evita lista[lista.length]

String s = null;
s != null && s.length() > 0 // false
s != null & s.length() > 0 // NullPointerException
```

JavaScript · ?. e ??

```
const u = { nome: "Ana", endereco: null };
u.endereco?.cidade // undefined, sem erro

const qtd = 0;
qtd || 10           // 10 (o 0 se perdeu!)
qtd ?? 10           // 0 (só troca null/undefined)
```

Cuidado com efeitos colaterais (p. 316)

Em `(a > b) || (b++ / 3)`, o `b++` só acontece quando `a <= b`. Em Python, `0 or "padrão"` devolve "padrão": `and` e `or` devolvem um dos **operandos**.

Compreendi que o curto-circuito interrompe a avaliação quando o resultado já está definido. Em Java, o `&&` pode evitar acessar uma variável nula ou ultrapassar o limite de uma lista, enquanto o `&` avalia as duas condições. Em JavaScript, o `??` substitui apenas `null` ou `undefined`, preservando valores como zero. Também percebi que operações com efeitos colaterais podem não acontecer quando a segunda expressão é ignorada.

13 - Slide

Sentenças de atribuição

C · a atribuição é uma expressão (p. 319-320)

```
sum = count = 0;           // múltiplos alvos
a = b + (c = d / b) - 1;    // exemplo do livro

while ((ch = getchar()) != EOF) { ... }

if (x = y) { ... } // compila! atribui e testa y
```

Python · atribuição múltipla (p. 320)

```
a, b = b, a                # troca sem temporária
if (n := len(dados)) > 5: ... # morsa (Python 3.8+)
```

Linguagem	Atribuição é expressão?
C, C++, JavaScript	sim: <code>if (x = y)</code> compila
Java, C#	sim, mas o <code>if</code> exige booleano
Python	não, exceto o operador morsa <code>:=</code>
Go	não: nem <code>x++</code> é expressão
Rust	vale <code>()</code> : <code>if x = 5</code> não compila

Linguagens funcionais (p. 321)

`val` (ML) e `let` (F#, Rust) dão **nome** a um valor, que não muda.

Compreendi que a atribuição também pode funcionar como uma expressão em algumas linguagens. Em C, escrever `x = y` dentro do `if` atribui o valor de `y` a `x`, podendo causar um erro se a intenção era comparar. Python permite trocar valores com atribuição múltipla e utilizar o operador `:=` para atribuir dentro de uma expressão. Já Java exige que a condição do `if` seja booleana.

14 - Slide:

Atribuição de modo misto

Java - só alargamento (p. 322)

```
double d = 3;          // ok: alargamento
// int i = 3.7;        // erro: estreitamento
int i = (int) 3.7;      // 3 (cast explícito)
long l = i;            // ok

i += 3.7;              // COMPILA: cast escondido
System.out.println(i); // 6
```

C - coerção em qualquer direção

```
int i = 3.7;           // 3, em silêncio
char c = 300;          // 44 (só um aviso)
unsigned int u = -1;    // 4294967295
```

Armadilha clássica

`double taxa = acertos / total * 100;` com inteiros dá **0.0**: a divisão inteira acontece **antes** do alargamento para double.

Compreendi que atribuições entre tipos diferentes podem alterar o valor armazenado. Em Java, o estreitamento normalmente exige uma conversão explícita, mas operadores como `+=` podem realizar essa conversão automaticamente. Em C, algumas conversões acontecem silenciosamente. Também percebi que armazenar uma divisão de inteiros em uma variável double não recupera as casas decimais, pois a divisão inteira ocorre antes da conversão.

15 - Slide:

Estruturas de controle: quantas são necessárias?

Sequência

uma sentença após a outra

```
a = 1;
b = a + 2;
```

Seleção

escolher um caminho

```
if (x > 0) ...
else ...
```

Iteração

repetir enquanto uma condição vale

```
while (x > 0)
    x--;
```

Böhm e Jacopini (1966): sequência, seleção e laço lógico bastam para qualquer algoritmo. O goto não é necessário.

Por que então tantas outras (`for`, `switch`, `do`, `foreach`)? **Facilidade de escrita.** O limite é a **legibilidade**: quanto maior a linguagem, menos do código alheio você reconhece (p. 329).

Compreendi que sequência, seleção e repetição são suficientes para representar qualquer algoritmo, sem precisar utilizar goto. A sequência executa uma instrução após a outra, a seleção escolhe um caminho e a repetição executa instruções enquanto uma condição permitir. Outras estruturas, como `for`, `switch` e `foreach`, facilitam a escrita, mas também é importante manter a legibilidade do código.

16 - Slide:

Seleção de dois caminhos: o "else pendente"

C - a indentação engana (p. 331)

```
int sum = 0, count = 5, result = -1;
if (sum == 0)
    if (count == 0)
        result = 0;
else
    result = 1; // de qual if é este else?
printf("%d\n", result); // 1
```

Regra de Java, C, C++ e C#

O else pertence ao if **mais próximo** que ainda não tem else. A indentação é ignorada (exceto em Python e F#).

Chaves obrigatórias eliminam o problema: Perl, Go e Rust (p. 332).

Seletores que devolvem valor

```
let y = if x > 0 { x } else { 2 * x }; // Rust: if é uma expressão (como em F#, p. 334)
y = x if x > 0 else 2 * x             # Python: expressão condicional (p. 306)
```

Compreendi que, em linguagens como C e Java, o else pertence ao if mais próximo que ainda não possui um else, independentemente da indentação. Isso pode causar uma interpretação errada do código, por isso utilizar chaves deixa os blocos claros. Também percebi que Rust e Python permitem usar uma condição para escolher um valor e atribuí-lo a uma variável.

17 - Slide:

Seleção múltipla: do switch de C ao match

C - sem break, o controle "cai" (p. 335)

```
switch (index) {
    case 1:
    case 3: odd += 1;
    case 2:
    case 4: even += 1;
    default: printf("Error\n");
}
// index = 1: odd, even E erro!
```

Java 14+ - switch como expressão

```
String tipo = switch (dia) {
    case SAB, DOM -> "fim de semana";
    case SEG, TER, QUA,
    QUI, SEX -> "dia útil";
}; // sem break, sem cair;
// exaustivo para enum
```

Rust - match exaustivo

```
match n {
    i32::MIN..=-1 => "negativo",
    0 => "zero",
    1..=9 => "um dígito",
    _ => "vários dígitos",
}
// sem o _ : não compila
```

Go: cada caso termina sozinho, e cair exige `fallthrough`. C#: cair implicitamente é proibido (p. 336). Python 3.10: match com padrões e guardas.

Compreendi que a seleção múltipla permite escolher entre vários caminhos. Em C, a ausência de break pode fazer a execução continuar pelos próximos casos, causando resultados inesperados. No Java, o switch com setas evita essa continuidade e pode retornar um valor. Já o match de Rust exige que todos os casos possíveis sejam contemplados, ajudando a evitar situações sem tratamento.

18 - Slide:

Laços controlados por contador

C - o for é um laço lógico disfarçado

```
for (int i = 0; i < 10; i++) {  
    printf("%d ", i);  
    i += 2;           // altera a contagem  
}  
  
for (; i <= 100; ) // expressões opcionais  
    soma += i++;
```

Python - o for percorre um iterável

```
for i in range(0, 10):  
    print(i, end=" ")  
    i += 2           # NÃO muda nada  
                    # 0 1 2 3 4 5 6 7 8 9  
  
for i in range(10, 0, -3): # 10 7 4 1  
    ...
```

Questões de projeto (p. 342)

Qual é o tipo e o escopo da variável? Posso alterá-la no corpo? Os parâmetros são avaliados uma vez ou a cada volta?

Compreendi que o for funciona de formas diferentes conforme a linguagem. Em C, alterar o contador dentro do laço interfere nas próximas repetições, como no exemplo que imprime 0, 3, 6 e 9. Em Python, o for percorre um iterável, e alterar a variável no corpo não muda o próximo valor fornecido pelo range. Para mudar a sequência, é possível definir o passo no próprio range.

19 - Slide:

Laços lógicos, saídas e iteradores

Java - break rotulado (p. 348)

```
externo:  
for (int i = 0; i < m.length; i++)  
    for (int j = 0; j < m[i].length; j++)  
        if (m[i][j] < 0) {  
            System.out.println(i + ", " + j);  
            break externo; // sai dos DOIS laços  
        }
```

JavaScript - for...in x for...of

```
const notas = [10, 20, 30];  
let total = 0;  
for (const i in notas) total += i;  
console.log(total); // "0012": chaves!  
  
total = 0;  
for (const v of notas) total += v;  
console.log(total); // 60: valores
```

Zune (2008) - o laço lógico sem progresso

```
while (dias > 365) {  
    if (bissexto(ano)) {  
        if (dias > 366) { dias -= 366; ano++; }  
        // dias == 366: nada muda, laço infinito!  
    } else { dias -= 365; ano++; }  
}
```

Pós-teste (do-while) executa **ao menos uma vez**, e isso é fácil de esquecer (p. 347).

Compreendi que os laços precisam avançar em direção à condição de saída para evitar repetições infinitas. Em Java, o break rotulado permite sair de vários laços aninhados de uma vez. Em JavaScript, o for...in percorre as chaves, enquanto o for...of percorre os valores, o que pode mudar o resultado de uma soma. Também percebi que o do-while executa o bloco pelo menos uma vez, pois verifica a condição depois.

20 - Slide:

Desvio incondicional: o goto

"goto fail" (Apple, 2014) - versão simplificada

```
int valida_certificado(void) {
    int err;
    if ((err = verifica_hash()) != 0)
        goto fail;
    goto fail;           // duplicada: desvia SEMPRE
    if ((err = verifica_assinatura()) != 0)
        goto fail;       // nunca é alcançada
fail:
    return err;          // 0 = "certificado válido"
}
```

Dijkstra (1968)

"Go To Statement Considered Harmful"

Quem tem goto hoje

C, C++, C#, Go. Java **não** tem (p. 353)

Gotos restritos

break, continue e return melhoram a legibilidade (p. 353)

Com chaves obrigatórias (Go, Rust) ou com gcc -Wall (aviso de indentação enganosa), o bug teria sido pego.

Compreendi que o goto desvia a execução diretamente para outro ponto do código, podendo dificultar o entendimento do fluxo. No exemplo, um goto duplicado faz a verificação da assinatura ser ignorada e pode considerar válido um certificado que não foi totalmente verificado. Utilizar chaves e avisos do compilador ajuda a identificar esse tipo de erro, enquanto break, continue e return permitem desvios mais controlados.

21 - Slide:

Comandos protegidos de Dijkstra

```
if x >= y -> max := x
[] y >= x -> max := y
fi
```

Exemplo do livro (p. 354): se $x == y$, qualquer ramo serve.

- Todas as **guardas** são avaliadas
- Várias verdadeiras: escolha **não determinística**
- Nenhuma verdadeira: erro (no if) ou fim do laço (no do ... od)
- Base do **CSP** de Hoare (1978) e do select de Go

Go - select, herdeiro dos comandos protegidos

```
a <- "a"      // os dois canais
b <- "b"      // estão prontos

select {      // Go sorteia uma guarda
case x := <-a:
    contagem[x]++
case x := <-b:
    contagem[x]++
}

// 1000 repetições:
// map[a:505 b:495] (muda a cada execução)
```

Compreendi que os comandos protegidos utilizam condições chamadas guardas para definir quais caminhos podem ser executados. Quando várias guardas são verdadeiras, qualquer um desses caminhos pode ser escolhido, sem uma ordem fixa. O select de Go segue uma ideia semelhante: quando vários canais estão prontos, escolhe uma das operações, podendo gerar resultados diferentes a cada execução.

22 - Slide:

Exercício em duplas: encontre o bug (1/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

1 - JavaScript

```
const precos = [10, 20, 30];
let total = 0;
for (const p in precos)
  total += p;
console.log("Total: " + total);
```

2 - Python

```
def desconto(preco, d=None):
    d = d or 10    # padrão: 10%
    return preco * (100 - d) / 100

print(desconto(200, 0))
```

1 - JavaScript:

a) A saída será "Total: 0012".

b) O for...in percorre os índices do array como texto. Por isso, o operador += concatena os índices ao total, em vez de somar os preços.

c) Substituir for...in por for...of, que percorre os valores do array. Assim, a saída será "Total: 60".

2 - Python:

a) A saída será 180.0.

b) O zero é considerado falso, então d or 10 substitui o desconto informado por 10, aplicando 10% de desconto.

c) Substituir d = d or 10 por d = 10 if d is None else d. Assim, o desconto padrão será aplicado somente quando d for None, e desconto(200, 0) retornará 200.0.

23 - Slide:

Exercício: encontre o bug (2/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

3 - Java

```
int dia = 2; String nome = "";
switch (dia) {
    case 1: nome = "domingo";
    case 2: nome = "segunda";
    case 3: nome = "terça";
    default: nome = "inválido";
}
System.out.println(nome);
```

4 - C

```
int saldo = 100, saque = 50;
if (saque > saldo);
    printf("saldo insuficiente\n");
printf("fim\n");
```

3 - Java:

- a) A saída será “inválido”.
- b) A ausência de break faz a execução continuar pelos casos seguintes e chegar ao default, que substitui o valor de nome por “inválido”.
- c) Adicionar break ao final de cada case. Assim, para dia = 2, a saída será “segunda”.

4 - C:

- a) A saída será “saldo insuficiente” e depois “fim”, em linhas separadas.
- b) O ponto e vírgula logo após o if encerra a instrução com um corpo vazio. Assim, a mensagem “saldo insuficiente” é impressa independentemente da condição.
- c) Remover o ponto e vírgula após o if e colocar a impressão da mensagem entre chaves. Como o saque de 50 não ultrapassa o saldo de 100, a saída será apenas “fim”.

24 - Slide:

Exercício: encontre o bug (3/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

5 - Go

```
x := 10
if x > 5 {
    x := x * 2
    fmt.Println("dentro:", x)
}
fmt.Println("fora:", x)
```

6 - Java

```
int acertos = 7, total = 10;
double taxa = acertos / total * 100;
System.out.println(taxa + "%");
```

Arquivos: exemplos/10_exercicio (q1.js ... Q6.java) - execute em onecompiler.com

5 - Go:

- a) A saída será “dentro: 20” e depois “fora: 10”, em linhas separadas.
- b) O uso de `:=` dentro do `if` declara uma nova variável `x`, que oculta a variável externa. Por isso, o `x` de fora continua valendo 10.
- c) Substituir `x := x * 2` por `x = x * 2`. Assim, a variável externa será alterada, e as duas impressões mostrarão 20.

6 - Java:

- a) A saída será “0.0%”.
- b) Como `acertos` e `total` são inteiros, `7 / 10` realiza uma divisão inteira e resulta em zero. A conversão para `double` acontece somente depois do cálculo.
- c) Converter um dos operandos antes da divisão, utilizando `double taxa = (double) acertos / total * 100`. Assim, a saída será “70.0%”.

25 - Slide:

Síntese: armadilhas e como as linguagens ajudam

Tema	Armadilha	Como as linguagens ajudam
Precedência	<code>-2 ** 2, 1 + 2 + "3"</code>	parênteses; JS obriga <code>(-2) ** 2</code>
Operandos	<code>a + f()</code> com efeito colateral (C, Go)	Java fixa a ordem; funções puras
Conversões	coerção silenciosa (C, JS, PHP)	Go e Rust exigem conversão explícita; <code>==</code>
Booleanos	<code>0, "", []</code> são falsos; <code>a > b > c</code> em C	<code>if</code> só com booleano (Java, Go, Rust); <code>??</code>
Atribuição	<code>if (x = y)</code>	atribuição não é expressão (Python, Go, Rust)
Seleção	<code>switch</code> sem <code>break</code> ; <code>else</code> pendente	<code>-></code> , <code>match</code> exaustivo, chaves obrigatórias
Laços	<code>for...in</code> no JS; laço sem progresso	<code>for...of</code> , iteradores, <code>break</code> rotulado
Desvio	<code>goto</code> sem restrição	<code>break</code> , <code>continue</code> , <code>return</code> : gotos restritos

Compreendi que muitos erros surgem de detalhes nas regras das linguagens, como conversões automáticas, ordem de avaliação, atribuições em condições e `switch` sem `break`. Recursos como parênteses, conversões explícitas, condições booleanas e blocos bem definidos ajudam a evitar essas armadilhas. Também percebi que conhecer as regras da linguagem e escrever um código claro é essencial para manter o fluxo de execução correto.